DEVELOPMENT REPORT: THE IMPOSTOR GAME

Authors: Andrés Rey Luna and Javier García Palomares

Project Nature: Real-Time Multiplayer Entertainment Web Application

1. Introduction and Project Origin

1.1. Context and Problem Statement

At the beginning of the current academic year, social deduction and hidden-word party games experienced a massive surge in popularity, becoming a staple at social gatherings and pre-parties among friends. However, the traditional mechanics of these board games or group activities suffered from a critical limitation: **the need for a dedicated moderator**.

The person responsible for managing the game (assigning roles, choosing words, and determining who the impostor was) was automatically excluded from the fun, as they held all the confidential information. Over time, several mobile applications entered the market to automate this process, but they came with severe drawbacks: they were heavily oversaturated with intrusive advertising or required every single player to download heavy software onto their devices.

1.2. Motivation of the Authors

Faced with this situation, the project authors, **Andrés Rey Luna and Javier García Palomares**, identified a clear opportunity for improvement. The original idea was born out of a personal need: they wanted to enjoy this game format within their own group of friends—which was often small—without forcing anyone to sacrifice themselves as a moderator and without enduring the constant interruptions of ads found in commercial platforms.

2. Game Definition and Concept

The Impostor Game is a real-time, hidden-screen local multiplayer party game. The core gameplay revolves around communication, deception, and deduction:

- **Role Assignment:** When a match begins, the system randomly selects a word pack, assigning a common word to the majority (the Citizens) and a subtly different decoy word to the Impostor.
- **Discussion Phase:** Players take turns describing their word using ambiguous clues to identify each other without revealing the exact term out loud.
- **The Core Catch:** The Impostor does not initially know they are the Impostor, which naturally creates highly entertaining situations of organic confusion and double bluffs.
- **Resolution:** Through an integrated dynamic voting system, the software manages player eliminations and automatically evaluates win conditions for both sides.

3. Architectural Decisions: The Choice of a Web Environment

During the initial design phase, Andrés and Javier evaluated the possibility of developing a native mobile application (iOS/Android). However, this was quickly ruled out due to the high financial costs of developer distribution licenses, the complexity of compiling for different platforms, and the friction of requiring guests to download and install an app in the middle of a party.

Considering the scope of the project—a game built for instant, local consumption—the authors mutually agreed on a **Single Page Application (SPA) Web Architecture**. This way, the host only needs to run the server on their machine, and the rest of the participants can instantly join by scanning a QR code using their smartphones' native web browsers, completely removing any barrier to entry.

4. Tech Stack and Technical Justification

The selection of the technological ecosystem was a highly collaborative process where both Javier and Andrés contributed ideas and solutions to meet the game's technical requirements: in-memory data persistence, real-time synchronization, and seamless local network (LAN) discoverability.

The implemented technologies are detailed below:

4.1. Backend and Server

Node.js: Proposed as the server runtime environment due to its asynchronous, event-driven nature, making it ideal for handling multiple concurrent connections with minimal resource consumption.

Express: Implemented to structure the HTTP server. Its lightweight design allowed the authors to serve the client's static files (HTML, CSS, JS) efficiently and centrally without adding unnecessary layers of complexity.

4.2. Real-Time Synchronization

Socket.io (WebSockets): One of the key developmental decisions. To avoid the performance overhead caused by constant HTTP requests (*polling*), the authors opted to open a permanent, bidirectional communication channel between the clients and the server. This ensures that player votes are reflected instantly on the host's screen and that screen transitions occur simultaneously and atomically across all connected devices.

4.3. Local Network Infrastructure and User Experience (UX)

internal-ip: Integrated into the backend to dynamically resolve the host machine's private IP address within the local Wi-Fi network.

qrcode-terminal: A brilliant idea to optimize user experience. This module transforms the server's IP address into a QR code rendered directly in the terminal console. Consequently, invited friends simply need to point their phone cameras at the host's screen to join, ingeniously solving the tedious task of typing manual IPs and ports.

4.4. Frontend (Client Interface)

Advanced HTML5 & CSS3: Designed a fully responsive layout optimized for mobile screens. Native CSS3 animations via `@keyframes` were implemented to provide visual dynamism and smooth screen transitions.

Vanilla JavaScript (ES6+): Instead of overloading the client side with heavy frameworks like React or Vue, Andrés and Javier decided to use pure JavaScript for DOM manipulation. This guarantees that the game loads instantly, even on older smartphones or under congested home Wi-Fi networks.

5. Conclusion and Future Work

5.1. Conclusion

The development of *The Impostor Game* has successfully achieved the goals originally set by its creators. By blending modern web technologies with an event-driven architecture, the role of the moderator has been completely automated. This provides a clean, agile, free, and ad-free alternative to spice up social gatherings. The project proves that it is entirely possible to create a fluid and engaging multiplayer experience with minimal resource overhead and without forcing users to install third-party software.

5.2. Future Work and Upcoming Milestones

The scalable nature of the current architecture (Node.js + Socket.io) opens the door to ambitious upgrades. The next steps of development will focus on the following milestones:

- **Global Cloud Deployment (Transitioning from LAN to WAN):** The most immediate priority is launching the application on the Internet. Currently, the game requires participants to share the same local Wi-Fi network. The goal is to deploy the backend to a cloud platform (such as Render, AWS, or Heroku) to make it publicly accessible via a standard web domain.
- **International Connectivity (Poland - Spain):** This leap to the global network stems from a personal and logistical need of the authors. Since they are currently residing temporarily in **Poland**, deploying the game to the internet will allow Andrés and Javier to continue playing, testing, and sharing real-time sessions with their core group of friends back in **Spain**. This will permanently break the barrier of physical distance, turning a local party game into a cross-border social deduction platform.
- **Access System Redesign (Room Codes):** Upon leaving the local network environment, the internal IP detection module (`internal-ip`) will be replaced by a secure Room Code system. The server will generate a unique identifier for each match, allowing players from anywhere in the world to join securely by simply entering the code on the website.
- **Persistence and Customization:** The authors plan to integrate a lightweight database (such as MongoDB or PostgreSQL) to allow user registration, historical match statistics tracking (win percentages as an impostor vs. citizen), and an interactive dashboard where users can create and save their own custom word packs.

6. Visual Annex: Application Interface and Game Flow

Figure 1 (Server Console): Initialization of the Node.js backend. `internal-ip` resolves the address and qrcode-terminal displays the connection QR code.

Figure 2 (Access Screen): Clean, responsive login interface with a "JOIN" button to register via Socket.io.

Figure 3 (Waiting Room/Lobby): Host-controlled lobby with real-time player list, impostor selector, and "START GAME" button.

Figure 4 (Game View - Round 1): Citizen view with secret word ("PIZZA"), voting panel, and phase controls.

Figure 5 (Match Evolution - Round 2): State after elimination. Frank is updated to show his status (💀).

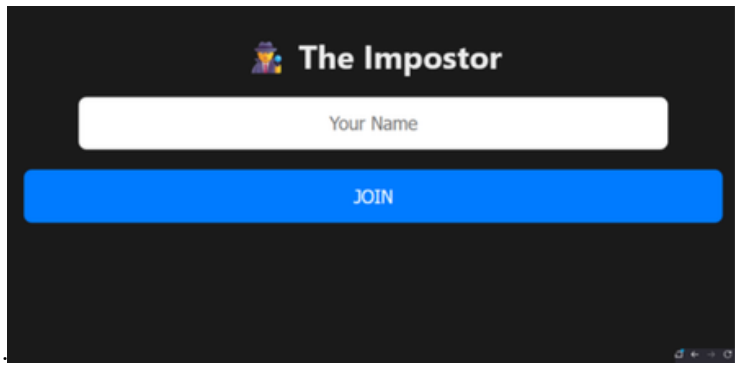
Figure 6 (Spectator Interface): View for an eliminated player. Voting buttons are disabled.

Figure 7 (Citizen Victory): Resolution screen triggered when all impostors are voted out.

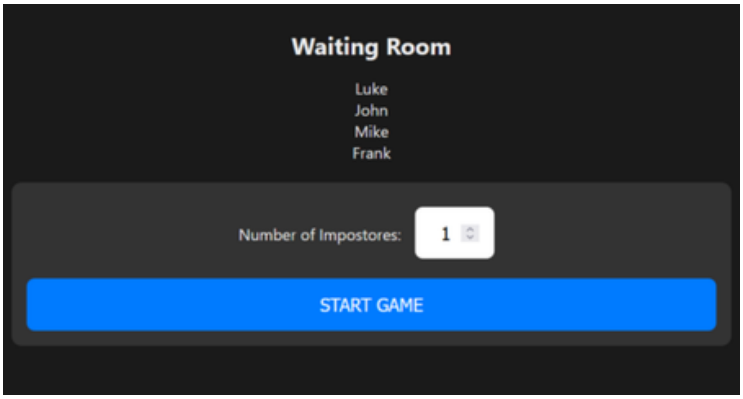
Figure 8 (Impostor Victory): Resolution screen for when impostors equal or outnumber citizens.

1.

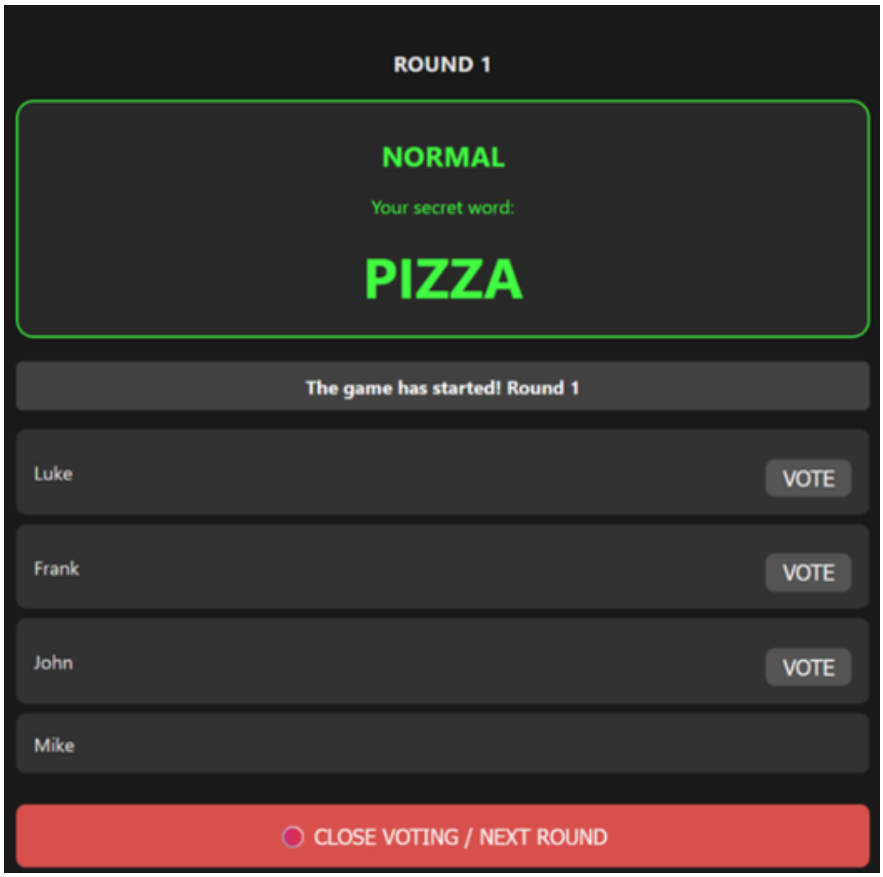
2



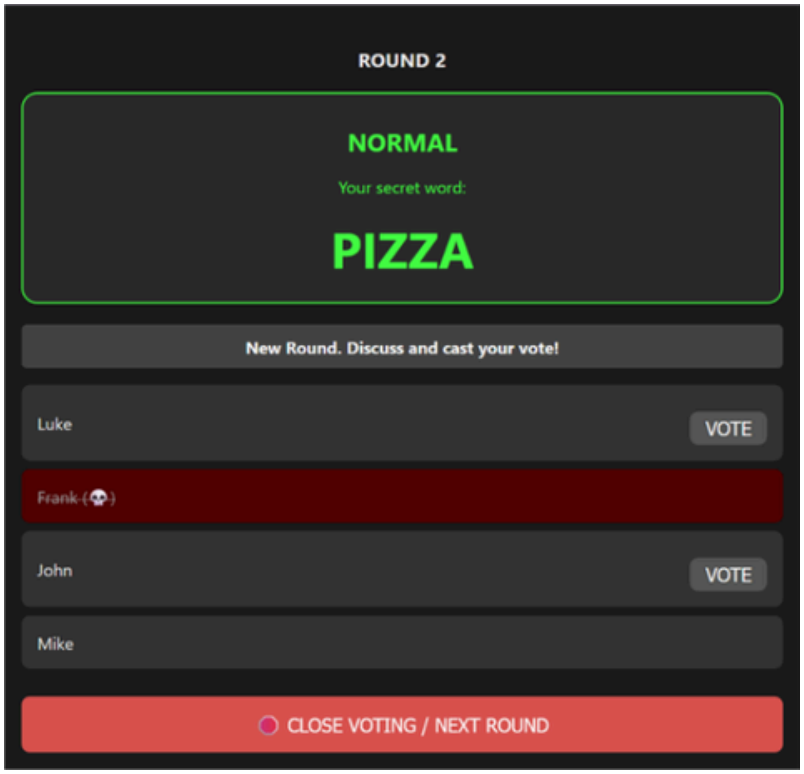
3.



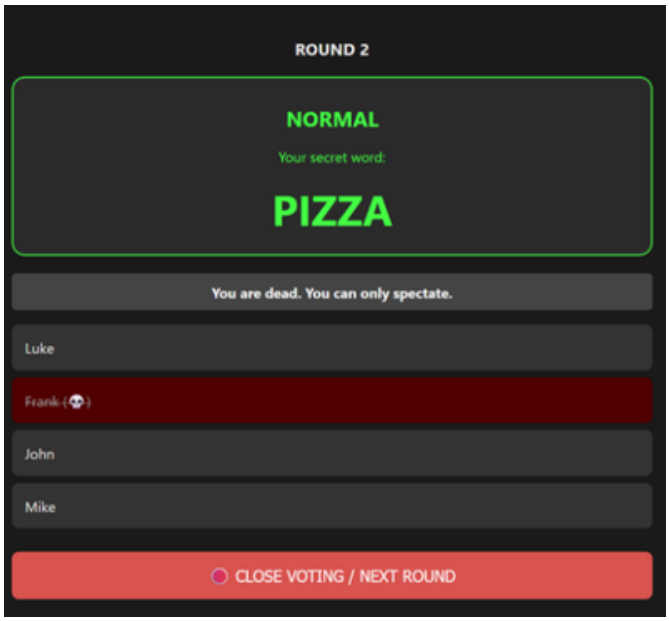
4.


ROUND 1
NORMAL
Your secret word:
PIZZA
The game has started! Round 1
Luke
VOTE
Frank
VOTE
John
VOTE
Mike
CLOSE VOTING / NEXT ROUND

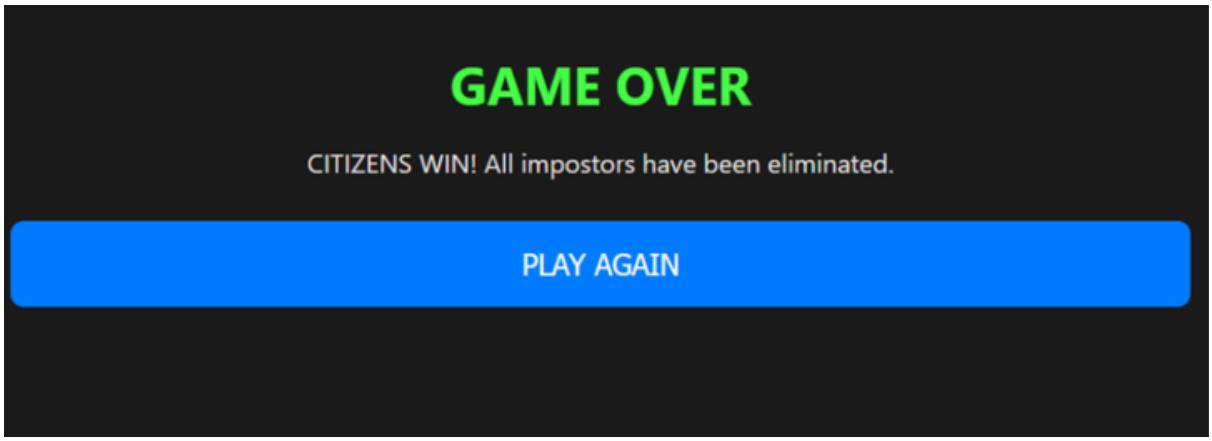
5.


ROUND 2
NORMAL
Your secret word:
PIZZA
New Round. Discuss and cast your vote!
Luke
VOTE
Frank
John
VOTE
Mike
CLOSE VOTING / NEXT ROUND

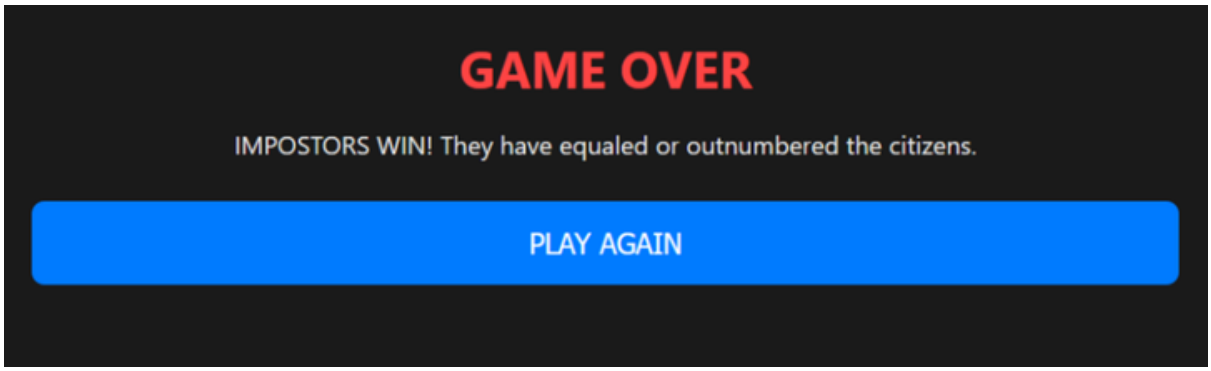
6.


ROUND 2
NORMAL
Your secret word:
PIZZA
You are dead. You can only spectate.
Luke
Frank (💀)
John
Mike
CLOSE VOTING / NEXT ROUND

7.


GAME OVER
CITIZENS WIN! All impostors have been eliminated.
PLAY AGAIN

8.


GAME OVER
IMPOSTORS WIN! They have equaled or outnumbered the citizens.
PLAY AGAIN